Cosmos EVM vs go-ethereum: Comprehensive Comparison

목차

- 개요
- 아키텍처 비교
- StateDB 구현 차이
- Transaction Processing
- Gas 계산 차이
- Precompile 처리
- Error Handling
- Performance 비교
- Trade-offs 분석
- 참고 자료

개요

Cosmos EVM과 go-ethereum(Geth)는 모두 EVM을 구현하지만, 기반 아키텍처와 설계 철학이 다릅니다. 이 문서에서는 두 구현체의 차이점을 상세히 분석합니다.

학습 목표:

- 두 구현체의 아키텍처 차이 이해
- StateDB 구현 방식 비교
- 각 구현체의 장단점 파악

예상 학습 시간: 3-4시간

아키텍처 비교

go-ethereum (Geth)

```
┌──────────────────────────────────────┐
│         Ethereum Network             │
│  (P2P, DevP2P, eth/66 protocol)      │
└──────────────────────────────────────┘
              │
              │
              ▼
┌──────────────────────────────────────┐
│       Consensus Layer                │
│  - PoW: Ethash                       │
│  - PoS: Beacon Chain (post-merge)    │
└──────────────────────────────────────┘
              │
              │
              ▼
┌──────────────────────────────────────┐
│       EVM Core                       │
│  - core/vm/evm.go                    │
│  - OpCode execution                  │
│  - Gas metering                      │
└──────────────────────────────────────┘
              │
              │
              ▼
┌──────────────────────────────────────┐
│       StateDB                        │
│  - core/state/statedb.go             │
│  - Merkle Patricia Trie              │
│  - LevelDB/Pebble                    │
└──────────────────────────────────────┘
```

Cosmos EVM

```
┌──────────────────────────────────────┐
│         Cosmos Network               │
│  (CometBFT P2P, Tendermint protocol) │
└──────────────────────────────────────┘
              │
              │
              ▼
┌──────────────────────────────────────┐
│       Consensus Layer                │
│  - CometBFT (BFT consensus)          │
│  - ABCI interface                    │
└──────────────────────────────────────┘
              │
              │
              ▼
┌──────────────────────────────────────┐
│       x/evm Module                   │
│  - Wraps go-ethereum EVM             │
│  - MsgEthereumTx handler             │
└──────────────────────────────────────┘
              │
              │
              ▼
┌──────────────────────────────────────┐
│       Custom StateDB                 │
│  - Maps to Cosmos SDK KVStore        │
│  - IAVL tree (not MPT)               │
│  - Custom storage layer              │
└──────────────────────────────────────┘
```

주요 차이점

측면	go-ethereum	Cosmos EVM
Consensus	PoW → PoS	CometBFT (BFT)
State Storage	MPT + LevelDB	IAVL + Cosmos KVStore
Network	DevP2P	CometBFT P2P
Account Model	Ethereum only	Ethereum + Cosmos
Transaction Format	RLP	Protobuf
Finality	Probabilistic	Instant (BFT)

StateDB 구현 차이

go-ethereum StateDB

```go
// core/state/statedb.go
type StateDB struct {
    db         Database
    prefetcher *triePrefetcher
    trie       Trie

    // State objects (accounts)
    stateObjects      map[common.Address]*stateObject
    stateObjectsDirty map[common.Address]struct{}

    // DB error
    dbErr error

    // Journal for reverting
    journal        *journal
    validRevisions []revision
    nextRevisionId int

    // Measurements
    AccountReads   time.Duration
    AccountHashes  time.Duration
    AccountUpdates time.Duration
    AccountCommits time.Duration
    StorageReads   time.Duration
    StorageHashes  time.Duration
    StorageUpdates time.Duration
    StorageCommits time.Duration

    // Snapshots
    snap snapshot.Snapshot

    // Per-transaction access list
    accessList *accessList

    // Transient storage
    transientStorage transientStorage
}
```

특징:

- ✅ **Merkle Patricia Trie**: 효율적인 Merkle proof
- ✅ **LevelDB/Pebble**: 최적화된 Key-Value 저장소
- ✅ **Snapshot**: 빠른 상태 접근
- ✅ **Prefetcher**: 병렬 읽기 최적화

Cosmos EVM StateDB

```go
// x/evm/keeper/statedb.go
type StateDB struct {
    keeper        Keeper
    ctx           sdk.Context

    // State objects
    stateObjects      map[common.Address]*stateObject
    stateObjectsDirty map[common.Address]struct{}

    // Journal
    journal        *journal
    validRevisions []revision
    nextRevisionId int

    // Precompiles
    precompiles map[common.Address]vm.PrecompiledContract

    // Logs
    logs    map[common.Hash][]*ethtypes.Log
    logSize uint

    // Refund counter
    refund uint64

    // EVM config
    evmDenom string
}
```

특징:

- ✅ **IAVL Tree**: Cosmos SDK 표준 Merkle tree
- ✅ **Cosmos KVStore**: 통합된 상태 관리
- ✅ **Context**: Cosmos SDK context 기반
- ❌ **No Snapshot**: IAVL 특성상 snapshot 부재
- ❌ **No Prefetcher**: 단순한 읽기 패턴

Storage 접근 비교

go-ethereum

```go
// core/state/statedb.go
func (s *StateDB) GetState(addr common.Address, hash common.Hash) common.Hash {
    stateObject := s.getStateObject(addr)
    if stateObject != nil {
        return stateObject.GetState(s.db, hash)
    }
    return common.Hash{}
}

// core/state/state_object.go
func (s *stateObject) GetState(db Database, key common.Hash) common.Hash {
    // Check dirty storage first
    value, dirty := s.dirtyStorage[key]
    if dirty {
        return value
    }

    // Check pending storage
    value, pending := s.pendingStorage[key]
    if pending {
        return value
    }

    // Check origin storage
    value, cached := s.originStorage[key]
    if cached {
        return value
    }

    // Load from trie
    enc, err := s.getTrie(db).TryGet(key.Bytes())
    if err != nil {
        s.setError(err)
        return common.Hash{}
    }

    value.SetBytes(enc)
    s.originStorage[key] = value
    return value
}
```

Cosmos EVM

```go
// x/evm/keeper/statedb.go
func (s *StateDB) GetState(addr common.Address, key common.Hash) common.Hash {
    obj := s.getStateObject(addr)
    if obj != nil {
        return obj.GetState(key)
    }
    return common.Hash{}
}

// x/evm/types/state_object.go
func (so *stateObject) GetState(key common.Hash) common.Hash {
    // Check dirty storage
    if value, ok := so.dirtyStorage[key]; ok {
        return value
    }

    // Check origin storage
    if value, ok := so.originStorage[key]; ok {
        return value
    }

    // Load from Cosmos KVStore
    store := so.db.ctx.KVStore(so.db.keeper.storeKey)
    bz := store.Get(StorageKey(so.address, key))

    if bz == nil {
        return common.Hash{}
    }

    var value common.Hash
    copy(value[:], bz)
    so.originStorage[key] = value
    return value
}
```

주요 차이:

- Geth: **MPT Trie** 사용, 복잡한 캐싱 레이어
- Cosmos EVM: **Direct KVStore** 접근, 단순한 구조

Transaction Processing

go-ethereum

```go
// core/state_processor.go
func (p *StateProcessor) Process(
    block *types.Block,
    statedb *state.StateDB,
    cfg vm.Config,
) (types.Receipts, []*types.Log, uint64, error) {
    var (
        receipts    types.Receipts
        usedGas     = new(uint64)
        header      = block.Header()
        blockHash   = block.Hash()
        blockNumber = block.Number()
        allLogs     []*types.Log
        gp          = new(GasPool).AddGas(block.GasLimit())
    )

    // Process transactions
    for i, tx := range block.Transactions() {
        statedb.Prepare(tx.Hash(), i)

        receipt, err := ApplyTransaction(
            p.config, p.bc, nil, gp, statedb,
            header, tx, usedGas, cfg,
        )
        if err != nil {
            return nil, nil, 0, fmt.Errorf("could not apply tx %d [%v]: %w", i, tx.Hash().Hex(), err)
        }

        receipts = append(receipts, receipt)
        allLogs = append(allLogs, receipt.Logs...)
    }

    return receipts, allLogs, *usedGas, nil
}
```

Cosmos EVM

```go
// x/evm/keeper/msg_server.go
func (k *Keeper) EthereumTx(
    goCtx context.Context,
    msg *types.MsgEthereumTx,
) (*types.MsgEthereumTxResponse, error) {
    ctx := sdk.UnwrapSDKContext(goCtx)

    // Create StateDB
    stateDB := NewStateDB(ctx, k)

    // Apply transaction
    result, err := k.ApplyTransaction(ctx, stateDB, msg.AsTransaction())
    if err != nil {
        return nil, err
    }

    // Commit state
    if err := stateDB.Commit(); err != nil {
        return nil, err
    }

    return &types.MsgEthereumTxResponse{
        Hash: msg.AsTransaction().Hash().Hex(),
        Logs: result.Logs,
        Ret:  result.Ret,
    }, nil
}
```

차이점:

- Geth: **Batch processing** (블록 단위)
- Cosmos EVM: **Single tx processing** (ABCI DeliverTx)

Gas 계산 차이

go-ethereum Gas

```
// core/vm/gas_table.go
const (
    GasQuickStep   uint64 = 2
    GasFastestStep uint64 = 3
    GasFastStep    uint64 = 5
    GasMidStep     uint64 = 8
    GasSlowStep    uint64 = 10
    GasExtStep     uint64 = 20

    SstoreSetGas    uint64 = 20000
    SstoreResetGas  uint64 = 5000
    SstoreClearGas  uint64 = 5000
    SstoreRefundGas uint64 = 15000
)

// EIP-2929 access lists
const (
    ColdAccountAccessCost = 2600
    ColdSloadCost         = 2100
    WarmStorageReadCost   = 100
)
```

Cosmos EVM Gas

```
// x/evm/types/params.go
type Params struct {
    // EVM denom
    EvmDenom string

    // Enable create
    EnableCreate bool

    // Enable call
    EnableCall bool

    // Extra EIPs
    ExtraEIPs []int64

    // Chain config
    ChainConfig ChainConfig
}

// Gas conversion
const (
    // 1 EVM gas = 1000 Cosmos SDK gas
    GasConversionRate = 1000
)

func EVMGasToCosmosGas(evmGas uint64) uint64 {
    return evmGas * GasConversionRate
}
```

차이점:

- Geth: **Pure EVM gas**
- Cosmos EVM: **EVM gas → Cosmos gas 변환**

Precompile 처리

go-ethereum Precompiles

```go
// core/vm/contracts.go
var PrecompiledContractsBerlin = map[common.Address]PrecompiledContract{
    common.BytesToAddress([]byte{1}): &ecrecover{},
    common.BytesToAddress([]byte{2}): &sha256hash{},
    common.BytesToAddress([]byte{3}): &ripemd160hash{},
    common.BytesToAddress([]byte{4}): &dataCopy{},
    common.BytesToAddress([]byte{5}): &bigModExp{},
    common.BytesToAddress([]byte{6}): &bn256AddIstanbul{},
    common.BytesToAddress([]byte{7}): &bn256ScalarMulIstanbul{},
    common.BytesToAddress([]byte{8}): &bn256PairingIstanbul{},
    common.BytesToAddress([]byte{9}): &blake2F{},
}

type PrecompiledContract interface {
    RequiredGas(input []byte) uint64
    Run(input []byte) ([]byte, error)
}
```

Cosmos EVM Custom Precompiles

```go
// x/evm/keeper/precompiles.go
var CosmosPrecompiles = map[common.Address]vm.PrecompiledContract{
    // Standard Ethereum precompiles
    common.BytesToAddress([]byte{1}): &ecrecover{},
    // ... (0x01-0x09)

    // Cosmos-specific precompiles
    common.BytesToAddress([]byte{0x08, 0x00}): &stakingPrecompile{},
    common.BytesToAddress([]byte{0x08, 0x01}): &bankPrecompile{},
    common.BytesToAddress([]byte{0x08, 0x02}): &govPrecompile{},
}

type stakingPrecompile struct {
    keeper stakingkeeper.Keeper
}

func (p *stakingPrecompile) Run(input []byte) ([]byte, error) {
    // Decode method selector
    if len(input) < 4 {
        return nil, errors.New("invalid input")
    }

    method := input[:4]

    switch string(method) {
    case "dele": // delegate
        return p.delegate(input[4:])
    case "unde": // undelegate
        return p.undelegate(input[4:])
    default:
        return nil, errors.New("unknown method")
    }
}
```

추가 기능:

- Cosmos EVM은 **Cosmos SDK 기능**을 EVM에 노출
- Staking, Governance, IBC 등을 스마트 컨트랙트에서 호출 가능

Error Handling

go-ethereum

```go
// core/vm/errors.go
var (
    ErrOutOfGas                 = errors.New("out of gas")
    ErrCodeStoreOutOfGas        = errors.New("contract creation code storage out of gas")
    ErrDepth                    = errors.New("max call depth exceeded")
    ErrInsufficientBalance      = errors.New("insufficient balance for transfer")
    ErrContractAddressCollision = errors.New("contract address collision")
    ErrExecutionReverted        = errors.New("execution reverted")
    ErrMaxCodeSizeExceeded      = errors.New("max code size exceeded")
    ErrInvalidJump              = errors.New("invalid jump destination")
    ErrWriteProtection          = errors.New("write protection")
    ErrReturnDataOutOfBounds    = errors.New("return data out of bounds")
    ErrGasUintOverflow          = errors.New("gas uint64 overflow")
)
```

Cosmos EVM

```go
// x/evm/types/errors.go
var (
    ErrInvalidState     = sdkerrors.Register(ModuleName, 2, "invalid state")
    ErrChainConfigNotFound = sdkerrors.Register(ModuleName, 3, "chain config not found")
    ErrInvalidChainConfig  = sdkerrors.Register(ModuleName, 4, "invalid chain config")
    ErrZeroAddress      = sdkerrors.Register(ModuleName, 5, "invalid zero address")
    ErrAccountNotFound  = sdkerrors.Register(ModuleName, 6, "account not found")
    ErrInvalidAmount    = sdkerrors.Register(ModuleName, 7, "invalid amount")
    ErrInvalidGasPrice  = sdkerrors.Register(ModuleName, 8, "invalid gas price")
    ErrCallDisabled     = sdkerrors.Register(ModuleName, 9, "EVM call is disabled")
    ErrCreateDisabled   = sdkerrors.Register(ModuleName, 10, "EVM create is disabled")
)
```

차이점:

- Geth: **EVM-specific errors**
- Cosmos EVM: **Cosmos SDK error wrapping** + EVM errors

Performance 비교

Benchmarks

Storage Access

Operation	Geth	Cosmos EVM	Ratio
SLOAD (cached)	100 gas	100 gas	1.0x
SLOAD (cold)	2,100 gas	2,100 gas	1.0x
SSTORE (new)	20,000 gas	20,000 gas	1.0x
SSTORE (update)	5,000 gas	5,000 gas	1.0x

EVM gas는 동일하지만, 실제 실행 시간은 다름:

- Geth: MPT + LevelDB (~0.1ms per SLOAD)
- Cosmos EVM: IAVL + KVStore (~0.3ms per SLOAD)

Transaction Throughput

Chain	TPS (Simple Transfer)	TPS (Complex Contract)
Geth (PoS)	~15-30	~10-15
Cosmos EVM	~50-100	~30-50

주요 요인:

- Geth: 블록 시간 12초
- Cosmos EVM: 블록 시간 5-7초 (CometBFT)

Trade-offs 분석

go-ethereum 장점

✅ Ethereum 호환성

- 완벽한 Ethereum 호환
- 기존 도구 그대로 사용 (MetaMask, Hardhat, etc.)

✅ 최적화

- MPT: 효율적인 Merkle proof
- Snapshot: 빠른 상태 접근
- Prefetcher: 병렬 읽기

✅ **성숙도**

- 수년간의 프로덕션 검증
- 방대한 커뮤니티

go-ethereum 단점

❌ **Finality**

- Probabilistic finality (재조직 가능)
- ~15-20 confirmations 필요

❌ **Consensus 제한**

- PoW → PoS 전환 복잡
- 다른 consensus 사용 어려움

Cosmos EVM 장점

✅ **Instant Finality**

- CometBFT BFT consensus
- 블록 확정 즉시

✅ **유연성**

- 다양한 consensus 옵션
- Cosmos SDK 기능 활용

✅ **Interoperability**

- IBC 프로토콜
- 다른 Cosmos 체인과 통신

✅ **Custom Precompiles**

- Staking, Governance 등 네이티브 통합

Cosmos EVM 단점

❌ **Storage 성능**

- IAVL tree: MPT보다 느림
- Snapshot 부재

❌ **Gas 변환**

- EVM gas ↔ Cosmos gas 변환 복잡

❌ **호환성 이슈**

- 일부 Ethereum 도구 호환 문제
- JSON-RPC 레이어 필요

권장 사항

go-ethereum 선택 시

사용 사례:

- 완벽한 Ethereum 호환 필요
- 기존 Ethereum 도구 활용
- PoW/PoS 사용

추천:

- 메인넷 fork
- Ethereum L2
- Private Ethereum network

Cosmos EVM 선택 시

사용 사례:

- Instant finality 필요
- Cosmos 생태계 통합
- IBC 활용
- Custom precompiles

추천:

- Cosmos app-chain
- EVM + Cosmos SDK 하이브리드
- Cross-chain DeFi

참고 자료

go-ethereum

- Geth GitHub
- Geth Documentation
- EVM Specification

Cosmos EVM

- Cosmos EVM GitHub
- Cosmos SDK
- CometBFT

작성자: B-Harvest Core Developer Candidate **버전**: 1.0 **최종 수정일**: 2025-01-XX